

要实现比赛要求的“月球仓储基地智能分拣搬运系统”，需按**“基础验证→核心功能实现→系统集成→优化与文档”**的逻辑拆解模块，每个模块依赖前序模块的输出，以下是具体拆解及分步实现方案（全程基于已有配置的 Ubuntu+ROS2 环境，重点复用赛事方提供的接口）：

一、模块拆解总览

按比赛流程（裁判任务→解析→识别→导航→抓取→放置→可视化）拆解为 9 个核心模块，模块间存在明确依赖关系（如“导航”依赖“视觉识别的货物位置”，“抓取”依赖“导航到达目标点”）：

1. 基础环境验证与启动模块
2. 大模型指令解析模块
3. 计算机视觉货物识别模块
4. Nav2 导航控制模块（含避障）
5. MoveIt2 机械臂抓取 / 放置模块
6. Foxglove 状态可视化模块
7. 系统集成与流程串联模块
8. 场景设计与性能优化模块
9. 技术文档编写与最终测试模块

二、分步实现方案（按优先级顺序）

模块 1：基础环境验证与启动（前置条件，1 天内完成）

目标：确认赛事方提供的 ROS2、Gazebo、Nav2、MoveIt2、Llama 大模型均能正常运行，排除环境故障。

核心依赖：赛事方预装的 Ubuntu 系统（密码ros）、已配置的仿真地图与机器人模型。

实现步骤：

1. 启动 Gazebo 仿真环境（验证机器人与场景加载）

打开终端，执行赛事方提供的仿真启动命令（文档 2/3 均提及）：

```
ros2 launch mybot gazebo_world.launch.py
```

- 验证标准：Gazebo 界面正常加载，显示 “移动机器人 + 6 自由度机械臂 + A/B/C 三个放置区 + 红 / 蓝立方体货物”，无崩溃或报错。
- 补充：若加载失败，检查 `df -h` 是否系统空间不足（需执行 `./expand_rootfs_up.sh` 扩容，文档 2 “文件系统扩容” 步骤）。

2. 启动 Nav2 导航系统（验证导航基础功能）

新建终端，执行导航启动命令：

```
ros2 launch bot_navigation nav_bringup_gazebo.launch.py
```

- 验证步骤：

- ① 在 RViz 界面点击「2D Pose Estimate」，拖动鼠标设置机器人初始位置（让雷达扫描轮廓与地图重合）；
- ② 点击「Nav2 Goal」，在地图上选择 A 区位置，发送目标位姿；
- ③ 观察机器人是否沿规划路径移动到 A 区，无碰撞（验证 `/scan` 雷达话题与 `/navigate_to_pose` 动作接口正常）。

3. 启动 MoveIt2 机械臂控制（验证机械臂基础功能）

新建终端，执行 MoveIt2 启动命令：

```
ros2 launch mybot my_moveit_rviz.launch.py
```

- 验证步骤：

- ① 在 MoveIt2 RViz 界面，拖动 “机械臂末端” 到任意位置；
- ② 点击「Plan & Execute」，观察机械臂是否按规划轨迹运动（验证 `/move_action` 动作接口与 `/compute_ik` 逆运动学服务正常）；
- ③ 测试夹爪：执行文档 2 中的夹爪控制脚本，验证夹爪开合：

```
cd ~/dev_ws/src/yzbot
./pick_on.sh # 夹爪张开
./pick_off.sh # 夹爪闭合
```

4. 启动 Llama 大模型服务端（验证大模型接口）

新建终端，执行文档 2 中的大模型服务启动命令（指定模型路径与端口）：

```
llama-server -m ~/llama.cpp/qwen2.5-coder-0.5b-instruct-q4_k_m.gguf -c 2048 --threads 8 --port 8081
```

- 验证步骤：新建终端测试对话，确认模型能响应：

```
cd ~/dev_ws/src/yzbot/tools_demo/tools_demo
python3 chat_interactive.py # 输入 “hello”，应返回类似 “Hello! How can I assist you?”
```

5. 验证 ROS2 话题 / 服务通信（确保模块间能传数据）

执行 `ros2 topic list`，确认以下关键话题存在：

- 相机话题： `/camera/image_raw`（视觉识别用）、 `/camera_rgbdepth/image_raw`（深度信息用）；
- 导航话题： `/plan`（规划路径）、 `/amcl_pose`（机器人定位）；
- 机械臂话题： `/arm_controller/state`（机械臂状态）、 `/gripper_controller/follow_joint_trajectory`（夹爪控制）。

模块 2：大模型指令解析模块（核心输入，1-2 天完成）

目标：将裁判的自然语言任务（如“搬运 2 个红色包裹到 A 区”）解析为机器可执行的结构化数据（颜色、数量、目标区域），并发布到 ROS2 话题。

核心依赖：模块 1 验证通过的 Llama 服务端、ROS2 节点编程。

关键接口：Llama 服务端（端口 8081）、自定义 ROS2 话题 `/llama_command`（发布结构化任务）。

实现步骤：

1. 编写大模型解析节点（Python）

在 `~/dev_ws/src/` 下新建功能包 `llama_parser`，创建节点 `command_parser.py`，核心逻辑：

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
import requests
import json
class LlamaCommandParser(Node):
```

```

def __init__(self):
    super().__init__("llama_command_parser")
    # 1. 创建话题发布者（发布解析后的任务）
    self.cmd_pub = self.create_publisher(String, "/llama_command", 10)
    # 2. 定义Llama服务端地址
    self.llama_url = "http://localhost:8081/completion"
    # 3. 模拟裁判输入（比赛时可改为接收裁判终端输入）
    self.get_user_input()
def get_user_input(self):
    while rclpy.ok():
        user_task = input("请输入裁判任务（如“搬运2个红色包裹到A区”）：")
        if user_task in ["quit", "exit"]:
            break
        # 4. 向Llama发送Prompt，要求结构化输出
        prompt = f"""你是机器人任务解析助手，仅需按JSON格式返回以下字段：
        - color: 货物颜色（red/blue）
        - num: 数量（整数）
        - area: 目标区域（A/B/C）
        任务：{user_task}，示例输出：{{"color":"red","num":2,"area":"A"}}"""

        # 5. 调用Llama服务端
        response = requests.post(
            self.llama_url,
            json={"prompt": prompt, "n_predict": 512, "temperature": 0.1}
        )
        # 6. 解析Llama返回结果，提取JSON
        llm_output = response.json()["content"].strip()
        self.get_logger().info(f"解析结果：{llm_output}")
        # 7. 发布到/llama_command话题
        cmd_msg = String()
        cmd_msg.data = llm_output
        self.cmd_pub.publish(cmd_msg)
def main(args=None):
    rclpy.init(args=args)
    node = LlamaCommandParser()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
if __name__ == "__main__":
    main()

```

2. 配置功能包与编译

在`package.xml`中添加依赖（`rclpy`、`std_msgs`、`requests`），在`setup.py`中配置入口：

```
entry_points={
    "console_scripts": [
        "command_parser = llama_parser.command_parser:main",
    ],
}
```

编译并 source 环境：

```
cd ~/dev_ws
colcon build --packages-select llama_parser
source install/local_setup.bash
```

3. 验证解析功能

- 启动 Llama 服务端（模块 1 的步骤 4）；
- 启动解析节点：`ros2 run llama_parser command_parser`；
- 输入裁判任务（如“搬运 3 个蓝色包裹到 C 区”），观察终端输出是否为 JSON 格式（如`{"color": "blue", "num": 3, "area": "C"}`）；
- 用`ros2 topic echo /llama_command`验证话题是否正常发布。

模块 3：计算机视觉货物识别模块（核心感知，2 天完成）

目标：实时识别 Gazebo 中红 / 蓝立方体货物的位置（x/y 坐标）与颜色，发布到 ROS2 话题，为导航和抓取提供目标位置。

核心依赖：模块 1 的相机话题、OpenCV、CvBridge（ROS2 与 OpenCV 图像转换）。

关键接口：相机话题/`camera/image_raw`（RGB 图像）、自定义话题/`cargo_detection`（发布识别结果）。

实现步骤：

1. 新建视觉识别功能包

在`~/dev_ws/src/`下新建`cargo_detection`功能包，依赖`rclpy`、`sensor_msgs`、`cv_bridge`、`opencv-python`：

```
ros2 pkg create --build-type ament_python cargo_detection --dependencies rclpy
sensor_msgs cv_bridge opencv-python numpy
```

2. 编写识别节点 (Python)

创建cargo_detector.py，核心逻辑：订阅相机图像→HSV 颜色分割→轮廓检测→计算中心点→发布结果：

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import String
from cv_bridge import CvBridge
import cv2
import numpy as np

class CargoDetector(Node):
    def __init__(self):
        super().__init__("cargo_detector")
        self.bridge = CvBridge()
        # 1. 订阅相机图像话题
        self.image_sub = self.create_subscription(
            Image, "/camera/image_raw", self.image_callback, 10
        )
        # 2. 发布识别结果（颜色+位置）
        self.detect_pub = self.create_publisher(String, "/cargo_detection", 10)
        # 3. 定义红/蓝HSV颜色范围（文档3提供参考值）
        self.color_ranges = {
            "red": [(0, 43, 46), (10, 255, 255)], # 红色HSV下限/上限
            "blue": [(100, 43, 46), (124, 255, 255)] # 蓝色HSV下限/上限
        }
    def image_callback(self, msg):
        # 4. ROS图像→OpenCV图像（BGR格式）
        cv_img = self.bridge.imgmsg_to_cv2(msg, "bgr8")
        # 5. BGR→HSV（颜色分割更稳定）
        hsv_img = cv2.cvtColor(cv_img, cv2.COLOR_BGR2HSV)
        # 6. 遍历红/蓝颜色，检测货物
        for color, (lower, upper) in self.color_ranges.items():
            # 颜色掩码（只保留目标颜色）
            mask = cv2.inRange(hsv_img, np.array(lower), np.array(upper))
            # 轮廓检测（过滤小噪声）
            contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
```

```

)
for cnt in contours:
    if cv2.contourArea(cnt) < 100: # 过滤小轮廓（噪声）
        continue
    # 7. 计算货物中心点坐标（图像坐标系）
    x, y, w, h = cv2.boundingRect(cnt)
    center_x = x + w/2
    center_y = y + h/2
    # 8. 转换为机器人坐标系（需根据相机内参校准，赛事方地图预设可简化为固定偏移）
    # 示例：假设图像x→机器人x，图像y→机器人y，偏移量(0.5, 0.3)（需实际调试）
    robot_x = center_x / 1000 + 0.5 # 图像像素→米
    robot_y = center_y / 1000 + 0.3
    # 9. 发布识别结果（JSON格式）
    detect_msg = String()
    detect_msg.data = json.dumps({
        "color": color,
        "position": {"x": robot_x, "y": robot_y},
        "timestamp": msg.header.stamp.sec
    })
    self.detect_pub.publish(detect_msg)
    # 10. 可视化（可选，方便调试）
    cv2.rectangle(cv_img, (x, y), (x+w, y+h), (0, 255, 0), 2)
    cv2.circle(cv_img, (int(center_x), int(center_y)), 5, (0, 0, 255), -1)
# 显示识别效果
cv2.imshow("Cargo Detection", cv_img)
cv2.waitKey(1)
def main(args=None):
    rclpy.init(args=args)
    node = CargoDetector()
    rclpy.spin(node)
    node.destroy_node()
    cv2.destroyAllWindows()
    rclpy.shutdown()
if __name__ == "__main__":
    main()

```

3. 编译与验证

- 编译功能包：`colcon build --packages-select cargo_detection`，source 环境；
- 启动 Gazebo（模块 1 步骤 1）；
- 启动识别节点：`ros2 run cargo_detection cargo_detector`；

- 验证标准：OpenCV 窗口显示相机画面，红 / 蓝货物被绿色矩形框选，中心点为红色圆点；用 `ros2 topic echo /cargo_detection` 查看是否输出颜色与机器人坐标系位置。

模块 4：Nav2 导航控制模块（核心移动，2 天完成）

目标：根据大模型解析的目标区域（A/B/C 区）和视觉识别的货物位置，控制机器人导航到目标点，同时避开 Gazebo 中的障碍物。

核心依赖：模块 2 的 `/llama_command`（目标区域）、模块 3 的 `/cargo_detection`（货物位置）、- Nav2 的 `/navigate_to_pose` 动作接口。

关键接口：Nav2 动作 `/navigate_to_pose`（类型 `nav2_msgs/action/NavigateToPose`）、雷达话题 `/scan`（避障数据）。

实现步骤：

1. 预设 A/B/C 区坐标

根据赛事方提供的 Gazebo 仿真地图，记录 A/B/C 区的固定坐标（示例，需实际调试）：

```
AREA_COORDS = {
    "A": {"x": 2.0, "y": 1.0, "yaw": 0.0}, # A区x/y/朝向
    "B": {"x": 0.0, "y": 3.0, "yaw": 1.57}, # B区（朝向90度）
    "C": {"x": -2.0, "y": 1.0, "yaw": 3.14} # C区（朝向180度）
}
```

2. 编写导航节点（Python）

在 `~/dev_ws/src/` 下新建 `nav_control` 功能包，创建 `navigator.py`，核心逻辑：订阅任务→获取目标位置→调用 Nav2 动作导航：

```
import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient
from std_msgs.msg import String
import json
from nav2_msgs.action import NavigateToPose
from geometry_msgs.msg import PoseStamped, Quaternion
from tf_transformations import quaternion_from_euler # 需安装：pip install tf-
transformations
class CargoNavigator(Node):
```



```

def __init__(self):
    super().__init__("cargo_navigator")
    # 1. 订阅任务 (/llama_command) 和货物位置 (/cargo_detection)
    self.task_sub = self.create_subscription(String, "/llama_command", self.task_callback, 10)
    self.cargo_sub = self.create_subscription(String, "/cargo_detection", self.cargo_callback, 10)

    # 2. 创建Nav2动作客户端
    self.nav_client = ActionClient(self, NavigateToPose, "/navigate_to_pose")

    # 3. 存储全局变量
    self.current_task = None # 当前任务 (color/num/area)
    self.cargo_pos = None # 当前检测到的货物位置

def task_callback(self, msg):
    # 4. 解析任务，更新当前任务
    self.current_task = json.loads(msg.data)
    self.get_logger().info(f"当前任务: {self.current_task}")

def cargo_callback(self, msg):
    # 5. 解析货物位置，仅保留当前任务颜色的货物
    cargo_data = json.loads(msg.data)
    if self.current_task and cargo_data["color"] == self.current_task["color"]:
        self.cargo_pos = cargo_data["position"]
        self.get_logger().info(f"目标货物位置: {self.cargo_pos}")

    # 6. 导航到货物位置 (若任务存在且未完成)
    self.navigate_to(self.cargo_pos["x"], self.cargo_pos["y"], 0.0)

def navigate_to(self, x, y, yaw):
    # 7. 等待Nav2动作服务就绪
    self.nav_client.wait_for_server()

    # 8. 构造目标位姿 (PoseStamped)
    goal_msg = NavigateToPose.Goal()
    pose = PoseStamped()
    pose.header.frame_id = "map" # 固定坐标系 (与Nav2一致)
    pose.header.stamp = self.get_clock().now().to_msg()

    # 位置
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.position.z = 0.0

    # 朝向 (四元数, 从欧拉角转换)
    quat = quaternion_from_euler(0.0, 0.0, yaw) # roll/pitch/yaw
    pose.pose.orientation = Quaternion(x=quat[0], y=quat[1], z=quat[2], w=quat[3])
    goal_msg.pose = pose

    # 9. 发送导航目标，等待结果
    self.get_logger().info(f"导航到: (x:{x}, y:{y})")

```

```

    future = self.nav_client.send_goal_async(goal_msg)
    future.add_done_callback(self.nav_result_callback)
def nav_result_callback(self, future):
    # 10. 处理导航结果
    goal_result = future.result()
    if goal_result.accepted:
        result_future = goal_result.get_result_async()
        result_future.add_done_callback(self.nav_done_callback)
    else:
        self.get_logger().error("导航目标被拒绝")
def nav_done_callback(self, future):
    result = future.result().result
    if result.status == 3: # Nav2成功状态码
        self.get_logger().info("导航到达目标点！ ")
        # 导航到货物后，触发抓取（后续模块集成）
        # 导航到目标区域后，触发放置（后续模块集成）
    else:
        self.get_logger().error(f"导航失败，状态码：{result.status}")
def main(args=None):
    rclpy.init(args=args)
    node = CargoNavigator()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
if __name__ == "__main__":
    main()

```

3. 避障测试

- 启动 Gazebo、Nav2（模块 1 步骤 1/2）；
- 启动导航节点：`ros2 run nav_control navigator`；
- 启动大模型解析节点（模块 2），输入任务（如“搬运 1 个红色包裹到 A 区”）；
- 在 Gazebo 中添加可移动物体（如额外的立方体），观察机器人是否绕开障碍物到达货物位置；
- 验证标准：`/scan`话题显示雷达检测到障碍物，机器人规划路径绕开，导航结果显示“到达目标点”。

模块 5：MoveIt2 机械臂抓取 / 放置模块（核心操作，2-3 天完成）

目标：导航到达货物位置后，控制机械臂抓取货物；导航到达目标区域后，控制机械臂放置货物。

核心依赖：模块 4 的导航完成信号、模块 3 的货物位置、MoveIt2 的运动规划接口。

关键接口：MoveIt2 动作/`move_action`（机械臂运动）、夹爪服务/`ATTACHLINK`（吸附货物）、/`DETACHLINK`（释放货物）。

实现步骤：

1. 编写机械臂控制节点（Python/C++，示例用 Python）

在`~/dev_ws/src/`下新建`arm_control`功能包，依赖`rclpy`、`moveit_ros_planning_interface`、`linkattacher_msgs`（吸附服务）：

```
import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient
from rclpy.service import Client
from moveit_msgs.action import MoveGroup
from linkattacher_msgs.srv import AttachLink, DetachLink
from geometry_msgs.msg import Pose
from std_msgs.msg import String
import json

class ArmGripperController(Node):
    def __init__(self):
        super().__init__("arm_gripper_controller")
        # 1. 创建MoveIt2动作客户端（控制机械臂）
        self.move_client = ActionClient(self, MoveGroup, "/move_action")
        # 2. 创建吸附/释放服务客户端
        self.attach_client = Client(self, AttachLink, "/ATTACHLINK")
        self.detach_client = Client(self, DetachLink, "/DETACHLINK")
        # 3. 订阅导航完成信号（后续集成时从导航节点接收）
        self.nav_done_sub = self.create_subscription(String, "/nav_done", self.nav_done_callback, 10)
        # 4. 订阅货物位置（/cargo_detection）
        self.cargo_sub = self.create_subscription(String, "/cargo_detection", self.cargo_callback, 10)
        # 5. 存储货物位置和当前任务
        self.cargo_pos = None
        self.current_task = None
    def cargo_callback(self, msg):
        # 解析货物位置
        self.cargo_pos = json.loads(msg.data)["position"]
    def nav_done_callback(self, msg):
```

```

# 导航完成：判断是“到货物位置”还是“到目标区域”
nav_type = msg.data # 如“cargo”（货物位置）、“area”（目标区域）
if nav_type == "cargo" and self.cargo_pos:
    self.grasp_cargo() # 抓取货物
elif nav_type == "area":
    self.place_cargo() # 放置货物
def grasp_cargo(self):
    # 6. 控制机械臂到货物抓取位置（基于货物位置，需调试高度z）
    target_pose = Pose()
    target_pose.position.x = self.cargo_pos["x"]
    target_pose.position.y = self.cargo_pos["y"]
    target_pose.position.z = 0.6 # 抓取高度（需调试，确保夹爪对准货物）
    target_pose.orientation.w = 1.0 # 朝向（默认）
    # 7. 调用MoveIt2动作，移动机械臂到抓取位置
    self.move_arm(target_pose, "arm") # “arm”是机械臂规划组名称
    # 8. 控制夹爪闭合（调用夹爪动作接口）
    self.control_gripper("close")
    # 9. 调用/ATTACHLINK服务，吸附货物（文档2接口）
    attach_req = AttachLink.Request()
    attach_req.model1_name = "six_arm" # 机械臂模型名
    attach_req.link1_name = "link6" # 机械臂末端link名
    attach_req.model2_name = f"{self.current_task['color']}_cube_1" # 货物模型名
    attach_req.link2_name = "link" # 货物link名
    self.attach_client.call_async(attach_req)
    self.get_logger().info("货物抓取完成！")
def place_cargo(self):
    # 10. 控制机械臂到放置位置（目标区域预设高度）
    target_pose = Pose()
    target_pose.position.x = AREA_COORDS[self.current_task["area"]]["x"]
    target_pose.position.y = AREA_COORDS[self.current_task["area"]]["y"]
    target_pose.position.z = 0.8 # 放置高度（高于地面）
    target_pose.orientation.w = 1.0
    self.move_arm(target_pose, "arm")
    # 11. 调用/DETACHLINK服务，释放货物
    detach_req = DetachLink.Request()
    detach_req.model1_name = "six_arm"
    detach_req.link1_name = "link6"
    detach_req.model2_name = f"{self.current_task['color']}_cube_1"
    detach_req.link2_name = "link"
    self.detach_client.call_async(detach_req)
    # 12. 控制夹爪张开

```

```

    self.control_gripper("open")
    self.get_logger().info("货物放置完成! ")
def move_arm(self, target_pose, group_name):
    # 等待MoveIt2服务就绪
    self.move_client.wait_for_server()
    # 构造MoveGroup动作目标
    goal_msg = MoveGroup.Goal()
    goal_msg.request.group_name = group_name
    goal_msg.request.target_pose = target_pose
    goal_msg.request.allow_replanning = True # 允许重规划
    # 发送目标并等待结果
    future = self.move_client.send_goal_async(goal_msg)
    future.add_done_callback(self.move_result_callback)
def move_result_callback(self, future):
    result = future.result()
    if result.accepted:
        self.get_logger().info("机械臂运动完成")
    else:
        self.get_logger().error("机械臂运动被拒绝")
def control_gripper(self, action):
    # 控制夹爪开合（调用/cargo_detection话题，或直接调用动作接口）
    # 示例：调用夹爪动作接口（/gripper_controller/follow_joint_trajectory）
    # 具体参数需参考赛事方提供的夹爪关节范围（如闭合时关节角度0.0，张开时0.5）
    pass
def main(args=None):
    rclpy.init(args=args)
    node = ArmGripperController()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
if __name__ == "__main__":
    main()

```

2. 抓取 / 放置测试

- 启动 Gazebo、MoveIt2（模块 1 步骤 1/3）；
- 启动机械臂控制节点：`ros2 run arm_control arm_controller`；
- 手动发送导航完成信号（如`ros2 topic pub /nav_done std_msgs/msg/String "data: 'cargo'"`），观察机械臂是否移动到货物位置并抓取；
-

再发送 `ros2 topic pub /nav_done std_msgs/msg/String "data: 'area'"`，观察是否移动到目标区域并放置；

- 验证标准：Gazebo 中机械臂夹爪对准货物→闭合→吸附货物→移动到目标区→释放，无碰撞。

模块 6：Foxglove 状态可视化模块（评分关键，1 天完成）

目标：按比赛要求，在 Foxglove 中显示 “指令解析、目标识别、导航路径、抓取状态、作业状态”，满足评分项 6。

核心依赖：前面所有模块的 ROS2 话题（`/llama_command`、`/cargo_detection`、`/plan`、`/grasp_status`等）、rosbridge（WebSocket 通信）。

关键接口：rosbridge_websocket、Foxglove 自定义布局。

实现步骤：

1. 安装并启动 rosbridge

rosbridge 用于将 ROS2 话题转为 WebSocket，供 Foxglove 访问：

```
sudo apt install ros-humble-rosbridge-suite # 若未预装
ros2 launch rosbridge_server rosbridge_websocket_launch.xml # 启动，默认端口9090
```

2. 配置 Foxglove 连接

- 打开 Foxglove Studio（桌面端或浏览器访问<https://app.foxglove.dev>）；
- 点击「Open Connection」→选择「Rosbridge」→输入 URL：`ws://[UbuntuIP]:9090`（UbuntuIP 可通过ifconfig查看，如192.168.1.100）；
- 点击「Open」，成功连接后会显示 ROS2 话题列表。

3. 自定义 Foxglove 布局（添加 5 个必需 Widget）

新建布局，按以下步骤添加 Widget 并绑定话题：

序号	Widget 名称	功能	绑定话题 / 数据来源
1	Text Log	指令解析显示	订阅 <code>/llama_command</code> ，显示任务 JSON
2	Image + Text	目标识别	图像： <code>/camera/image_raw</code> ；文本

			: /cargo_detection (颜色 + 位置)
3	3D Panel	导航路径	选择「Frame: map」，添加「Path」→绑定/plan (全局路径)
4	State Indicator	抓取状态	订阅自定义话题/grasp_status (发布 “success/fail” + 位置)
5	Text Log	作业状态反馈	订阅自定义话题/job_status (发布 “已抓取红色包裹” “已放置到 A 区” 等文本)

4. 验证可视化效果

- 启动所有模块（Gazebo、Nav2、大模型解析、视觉识别、导航、抓取）；
- 输入裁判任务，观察 Foxglove：

- ① Text Log 显示解析后的任务；
- ② Image 显示相机画面，Text 叠加货物颜色与位置；
- ③ 3D Panel 显示机器人实时导航路径；
- ④ State Indicator 在抓取成功时显示 “success”，失败时显示 “fail”；
- ⑤ Text Log 实时更新作业状态（如 “导航到红色货物位置” “抓取成功”）。

模块 7：系统集成与流程串联（核心，1-2 天完成）

目标：编写主控制节点，按比赛流程串联所有模块：

裁判任务→大模型解析→视觉识别货物→导航到货物→抓取→导航到目标区→放置→重复直到完成任务→结束

实现步骤:

1. 编写主控制节点 (main_controller.py)

在~/dev_ws/src/下新建system_integration功能包，核心逻辑:

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String
import json
import time

class MainController(Node):
    def __init__(self):
        super().__init__("main_controller")
        # 1. 订阅各模块信号
        self.task_sub = self.create_subscription(String, "/llama_command", self.task_callback, 10)
        self.nav_done_sub = self.create_subscription(String, "/nav_done", self.nav_done_callback, 10)
        self.grasp_done_sub = self.create_subscription(String, "/grasp_done", self.grasp_done_callback, 10)
        # 2. 发布控制信号
        self.nav_ctrl_pub = self.create_publisher(String, "/nav_control", 10) # 控制导航目标
        self.arm_ctrl_pub = self.create_publisher(String, "/arm_control", 10) # 控制抓取/放置
        self.job_status_pub = self.create_publisher(String, "/job_status", 10) # 作业状态
        # 3. 全局状态变量
        self.current_task = None # 当前任务 (color/num/area)
        self.completed_num = 0 # 已完成数量
        self.current_step = "wait_task" # 任务步骤: wait_task→nav_cargo→grasp→nav_area→place
    def task_callback(self, msg):
        # 接收任务，初始化状态
        self.current_task = json.loads(msg.data)
        self.completed_num = 0
        self.current_step = "nav_cargo"
        self.publish_job_status(f"开始任务: 搬运{self.current_task['num']}个{self.current_task['color']}包裹到{self.current_task['area']}区")
        # 触发第一次导航到货物
        self.publish_nav_control("cargo")
    def nav_done_callback(self, msg):
        # 导航完成，切换步骤
        nav_target = msg.data
```



```

if nav_target == "cargo" and self.current_step == "nav_cargo":
    self.current_step = "grasp"
    self.publish_arm_control("grasp") # 触发抓取
    self.publish_job_status("导航到货物位置，准备抓取")
elif nav_target == "area" and self.current_step == "nav_area":
    self.current_step = "place"
    self.publish_arm_control("place") # 触发放置
    self.publish_job_status("导航到目标区域，准备放置")
def grasp_done_callback(self, msg):
    # 抓取/放置完成，更新数量或结束任务
    action = msg.data
    if action == "grasp_success":
        self.current_step = "nav_area"
        self.publish_nav_control("area") # 导航到目标区域
        self.publish_job_status("抓取成功，导航到目标区域")
    elif action == "place_success":
        self.completed_num += 1
        self.publish_job_status(f"放置成功，已完成{self.completed_num}/{self.current_task['num']}个")
    # 判断是否完成所有任务
    if self.completed_num >= self.current_task["num"]:
        self.current_step = "wait_task"
        self.publish_job_status("所有任务完成！")
    else:
        self.current_step = "nav_cargo"
        self.publish_nav_control("cargo") # 继续抓取下一个
def publish_nav_control(self, target):
    # 发布导航控制信号 (cargo/area)
    msg = String()
    msg.data = target
    self.nav_ctrl_pub.publish(msg)
def publish_arm_control(self, action):
    # 发布机械臂控制信号 (grasp/place)
    msg = String()
    msg.data = action
    self.arm_ctrl_pub.publish(msg)
def publish_job_status(self, status):
    # 发布作业状态到Foxglove
    msg = String()
    msg.data = status
    self.job_status_pub.publish(msg)

```

```

        self.get_logger().info(status)
def main(args=None):
    rclpy.init(args=args)
    node = MainController()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
if __name__ == "__main__":
    main()

```

2. 修改前序模块适配主控制

- 导航节点（模块 4）：订阅/nav_control，接收“cargo/area”信号，切换导航目标（货物位置 / 目标区域）；
- 机械臂节点（模块 5）：订阅/arm_control，接收“grasp/place”信号，切换动作；
- 新增/grasp_done话题：机械臂抓取 / 放置完成后发布“grasp_success/place_success”。

3. 全流程测试

- 按顺序启动所有节点：

```

# 1. 基础环境
ros2 launch mybot gazebo_world.launch.py
ros2 launch bot_navigation nav_bringup_gazebo.launch.py
ros2 launch mybot my_moveit_rviz.launch.py
rosbridge_websocket_launch.xml
# 2. 核心模块
ros2 run llama_parser command_parser
ros2 run cargo_detection cargo_detector
ros2 run nav_control navigator
ros2 run arm_control arm_controller
# 3. 主控制
ros2 run system_integration main_controller
# 4. Foxglove
打开Foxglove并连接

```

- 输入裁判任务（如“搬运 2 个红色包裹到 A 区”），观察全流程：

① 大模型解析任务→Foxglove 显示指令；

② 视觉识别红色货物→导航到货物；

③ 抓取货物→导航到 A 区；

④ 放置货物→更新完成数量；

⑤ 重复步骤②-④直到完成 2 个→显示 “任务完成” ；

- 验证标准：5 分钟内完成任务，所有步骤在 Foxglove 中实时显示，无崩溃。

模块 8：场景设计与性能优化（提分项，1 天完成）

目标：构建复杂场景（提高评分项 7 “场景设计分”），优化系统稳定性与速度（确保 5 分钟内完成任务）。

实现步骤：

1. 场景复杂度提升

- 在 Gazebo 的 world 文件（如~/dev_ws/src/mybot_description/worlds/box.world）中添加元素：

① 斜坡：添加`<model name="ramp"><pose>1.0 0.0 0.0 0 0 0</pose><link><collision><geometry><box><size>1.0 0.5 0.3</size></box></geometry></collision></link></model></code>；`

② 可移动障碍物：添加 2-3 个额外的立方体（red_cube_2、blue_cube_2），设置为可移动（`<-allow_auto_disable>>false</allow_auto_disable>`）；

- 验证：机器人能在斜坡上行驶，绕开可移动障碍物。

2. 性能优化

- 视觉识别：降低图像分辨率（如从 1280x720 改为 640x480），减少延迟；
- 导航：调整 Nav2 参数（originbot_nav2.yaml），提高规划频率（`controller_frequency: 30`）；
- 机械臂：优化 MoveIt2 规划参数（减少规划时间，如`planning_time: 0.5`）；
- 测试：多次运行任务，确保完成时间 < 4 分钟（留 1 分钟冗余）。

模块 9：技术文档编写与最终测试（收尾，1 天完成）

目标：完成评分项 8 “技术文档”，全面测试系统稳定性。

实现步骤：

1. 编写技术文档（需包含以下内容）

- 系统架构图：模块划分（主控制、大模型解析、视觉识别等）、节点 / 话题 / 服务列表（用表格列出）；
- 核心算法创新点：如 “视觉识别的 HSV 动态阈值调整” “Nav2 路径优化（结合机械臂抓取范围）” “大模型 Prompt 工程（结构化输出约束）”；
- 关键代码片段：每个模块的核心代码（如大模型解析、导航调用、抓取控制）；
- 测试结果：抓取成功率（如 10 次测试成功 9 次）、导航成功率（10/10）、避障次数（2 次 / 任务）、任务完成时间（3 分 30 秒）；
- 问题与解决：如 “机械臂抓取失败→调整抓取高度 z 从 0.5 改为 0.6” “导航偏航→校准初始位姿”。

2. 最终测试（模拟比赛场景）

- 由队友扮演裁判，随机给出 3 个不同任务（如 “1 红 1 蓝到 B 区” “3 蓝到 C 区”）；
- 按比赛流程操作：启动所有节点→申请开始→计时→完成任务→举手示意结束；
- 记录每次任务的完成时间、成功率，根据问题最后调整（如抓取失败则优化 IK 参数）。

三、关键注意事项

1. **模块测试优先**：每个模块单独测试通过后再集成，避免集成后难以定位问题（如视觉识别失败会导致后续导航 / 抓取全错）；
2. **接口一致性**：所有模块的话题名称、消息格式需统一（如 JSON 字段大小写、坐标系 “map” 一致）；
3. **时间把控**：比赛限时 5 分钟，集成后需反复测试优化速度，避免因延迟超时；
4. **备份与恢复**：赛前备份 `dev_ws` 工作空间和仿真配置，避免环境损坏无法恢复。

按以上步骤实现，即可完成比赛要求的 “大模型 + 视觉 + 导航 + 机械臂 + 可视化” 全流程功能，覆盖所有评分项。